Introducing Shepherd, a reinforcement learning based rocket hovering model *

Andrei O. Vilcan
Radboud Universiteit
Houtlaan 4, Nijmegen, 6525XZ
andrei.vilcan@ru.nl

Abstract

We present Shepherd, a model-free actor-critic agent that learns to stabilize a rocket against the force of gravity in a simulated two-dimensional physics environment. The method we use to train the agent is the episodic REINFORCE algorithm, augmented with Generalized Advantage Estimation (GAE) and extended with curriculum learning, training on subsets of the final task with increasing difficulty. Our key finding is that for this task, GAE, which reduces the effective lookahead, is essential for stable training.

1 Introduction

Stabilizing a rocket in a constant hover at a particular location in space is a demanding continuous control problem - one that involves fine and rapid adjustments taking into consideration the effect of one's actions on position, velocity, angle, and angular velocity, in order to both counteract the force of gravity and keep the rocket in a stable position. In this work, the research question we address is as follows: Can a model-free policy-gradient actor-critic reinforcement-learning agent stabilize a rocket in a two-dimensional simulated gravitational environment? We further augment the REINFORCE algorithm (Williams, 1992) with Generalized Advantage Estimation (GAE) (Schulman et al., 2016) and train the agent with a difficulty-scaled curriculum, overall improving the agent's performance and arriving at a more robust policy.

2 Methods

2.1 Environment

First, we implement a two-dimensional rocket hovering Gymnax environment.

State The state of the system is fully described by 12 variables, one of which is time; the timestep is 0.02 seconds, and there is a maximum episode length of 1000 timesteps, or 20 seconds. The rocket's position and orientation are described by cartesian coordinates (x, y) and an angle θ respectively; its linear and angular velocity are described by $(\dot{x}, \dot{y})$ and ω respectively. The rocket has one main thruster placed on the bottom, which faces downwards, and thrust is described by a scalar τ. The thruster's gimbal (orientation relative to the rocket) is further described by an angle g. In addition, the rocket has two small thrusters near the top, one on either side, for balancing. The fuel remaining in the rocket is described by a variable f.

Actions The agent outputs 4 dimensional continuous actions in the range of [-1, 1], with each dimension corresponding to one of the following: the throttle of the main booster, the gimbal angle of

*Code available at https://github.com/andrei-vilcan/shepherd-hover.

the main booster, and the throttle of the left and right boosters. These are then mapped to the range [0, 1], in the case of the booster throttles, and to the range of [-0.3 rad, 0.3 rad] in the case of the gimbal.

Physics Throughout the whole environment, there is a constant downwards gravitational force exerted on the rocket equal to that present on Earth, of 9.81 m/s^2. The rocket has a height $h = 10$ m, a mass $m = 1000$ kg, and a moment of inertia $I = 10{,}000\,\text{kg}\cdot\text{m}^2$. The main thruster has a maximum thrust of $20{,}000$ N, whereas the side thrusters have a maximum thrust of 500 N, and they are placed 5 meters up from the center of the rocket, on each side. All thrusters influence the positional and rotational velocity of the rocket according to the equations below.

$$\ddot{x} = \frac{F_{\text{main}}\sin(\theta + g) + F_{\text{side}}\cos\theta}{m}, \qquad \ddot{y} = -g_{\text{grav}} + \frac{F_{\text{main}}\cos(\theta + g) + F_{\text{side}}\sin\theta}{m} \tag{1}$$

$$\ddot{\theta} = \frac{F_{\text{main}}\sin(g)\cdot d_m + F_{\text{side}}\cdot d_s}{I} \tag{2}$$

where $F_{\text{side}} = F_\ell - F_r$ is the net lateral force from the auxiliary thrusters, and d_m, d_s are the distances of the main engine and side thrusters from the center of mass.

Observation The agent observes an 11-dimensional vector as its input, which contains the state variables. The agent has perfect observability over these variables, and the positions, velocities, orientation, angular velocity, gimbal, and fuel are each normalized by constants c_x, c_y, c_v, c_ω, with each constant calculated according to a respective formula as a function of an offset factor defined in the curriculum.

$$\mathbf{o} = \left[\frac{x}{c_x},\ \frac{y}{c_y},\ \frac{\dot{x}}{c_v},\ \frac{\dot{y}}{c_v},\ \frac{\theta}{\pi},\ \frac{\omega}{c_\omega},\ \tau,\ \frac{g}{g_{\text{max}}},\ \ell,\ r,\ \frac{f}{f_{max}}\right] \tag{3}$$

Reward The reward the agent receives at each time step as a function of the state variables is calculated as the weighted sum of the exponential of the negative difference between the target value and the present value for the position, velocity, angle, and angular velocity. Generally, this means that being close to the origin (the target), having low velocity, being upright, and having a low angular velocity are rewarded in an exponentially decaying manner. Specifically, the reward is calculated according to the following equation,

$$r_t = \alpha_d \exp\left(\frac{-d_t}{5}\right) + \alpha_v \exp\left(\frac{-v_t}{5}\right) + \alpha_\theta \exp\left(\frac{-|\theta_t|}{0.5}\right) + \alpha_\omega \exp\left(\frac{-|\omega_t|}{1}\right) \tag{4}$$

where $d_t = \sqrt{x_t^2 + y_t^2}$ is the distance, $v_t = \sqrt{\dot{x}_t^2 + \dot{y}_t^2}$ is the velocity, and the reward factors $(\alpha_d, \alpha_v, \alpha_\theta, \alpha_\omega) = (2, 1, 1, 1)$ specify the importance of each measure in the calculation. The per-component reward curves are shown in Figure 2 in the Appendix.

2.2 REINFORCE Algorithm

Policy Network (Actor) The policy π generates actions by sampling from Gaussian distributions, whose means $\boldsymbol{\mu} \in \mathbb{R}^4$ are calculated as a function of the observation $\mathbf{o}$, through a multi-layer perceptron (MLP) with hidden layers of sizes $256, 128$, using the $\tanh$ activation function between layers, as well as a final $\tanh$, bounding outputs to $[-1, 1]$. Meanwhile, the logarithm of the standard deviations $\log\boldsymbol{\sigma}$ are learnable parameters as well and control the amount of exploration the agent can perform. Actions are sampled by calculating the value of the following variable.

$$\mathbf{a}_t = \text{clip}(\tilde{\mathbf{a}}_t,\ -1,\ 1), \qquad \tilde{\mathbf{a}}_t \sim \mathcal{N}\big(\boldsymbol{\mu}_\phi(\mathbf{o}_t),\ \text{diag}(\boldsymbol{\sigma}^2)\big) \tag{5}$$

where $\boldsymbol{\mu}_\phi(\mathbf{o}_t)$ is the MLP output, and $\boldsymbol{\sigma} = \exp(\log\boldsymbol{\sigma})$ with $\log\boldsymbol{\sigma}$ clipped to $[-2.0, 0.5]$.

Value Network (Critic) A separate network outputs a single scalar, calculating an estimate for the value of being in a certain state via a MLP with hidden layers of sizes $256, 128$, using $\tanh$ activation functions between layers.

Generalized Advantage Estimation We estimate the advantage function using the methodology described in Schulman et al. (2016). First, we obtain the TD residual of V with discount γ as $\delta_t = r_t + \gamma V(\mathbf{o}_{t+1}) - V(\mathbf{o}_t)$, where V is the estimated value of the state $\mathbf{o}_t$. Then, we calculate the GAE according to the following formula (Schulman et al., 2016, Eq. 16):

$$\hat{A}_t^{\mathrm{GAE}(\gamma,\lambda)} = \sum_{l=0}^{\infty} (\gamma\lambda)^l \, \delta_{t+l} \tag{6}$$

Note, setting $\lambda = 1$, we obtain the episodic REINFORCE policy gradient algorithm with the value function as a baseline (Williams, 1992). Comparatively, using $\lambda < 1$ we introduce a small amount of bias with the benefit of reduced variance of the advantage (Schulman et al., 2016).

Value Estimation After this, we must recover the returns; to obtain an estimate, we perform, as per the suggestion in (Schulman et al., 2016):

$$\hat{G}_t = \hat{A}_t + V_\psi(\mathbf{o}_t) \tag{7}$$

Policy Gradient The actor is trained by minimizing the REINFORCE policy loss (Williams, 1992): the product of the mean negative log-likelihood and the estimated advantage.

$$\mathcal{L}_\pi(\phi) = -\frac{1}{N} \sum_{i=1}^{N} \frac{1}{|\mathcal{T}_i|} \sum_{t \in \mathcal{T}_i} \log \pi_\phi(\mathbf{a}_t^{(i)} \mid \mathbf{o}_t^{(i)}) \, \hat{A}_t^{(i)} \tag{8}$$

Meanwhile, the critic is trained by minimizing the value loss function: the mean-square-error between predicted values and estimated returns.

$$\mathcal{L}_V(\psi) = \frac{1}{\sum_i |\mathcal{T}_i|} \sum_{i=1}^{N} \sum_{t \in \mathcal{T}_i} \left(V_\psi(\mathbf{o}_t^{(i)}) - \hat{G}_t^{(i)} \right)^2 \tag{9}$$

2.3 Curriculum Learning

Curriculum Stages To bring to a hover at a specific target a rocket that is dealing with arbitrary fluctuations in its position, velocity, orientation, and angular velocity, is certainly a more difficult task than to teach one that is already in the target state. In the first stage of the curriculum, we train the rocket to hover, starting in an upright position at the origin, and with zero (angular) velocity. In the second stage, using a scaling factor, we slowly increase the difficulty of the task until the maximum perturbations are as follows: maximum distance is $20\,\mathrm{m}$, velocity $10\,\mathrm{m/s}$, orientation 0.3 radians, and angular velocity 0.5 radians. In the third and final stage of the curriculum, the state variables are initialized according to maximum perturbation values. Each curriculum phase has $100{,}000$ episodes.

Difficulty Coefficient Using a progressively increasing difficulty coefficient during the second stage, we smoothly transition between the first and last stages. We scale the difficulty of the task by multiplying the maximum initial perturbation values by the output of the function $c(p) = \frac{1}{1+\exp(-e^e \cdot (p - 1/2))}$, where p is the normalized progress in this stage of the curriculum. This function is visualized in Figure 3 in the Appendix.

3 Results

We train the agent for $300{,}000$ episodes in batches of 16, resulting in the training curves in Figure 1.

3.1 Training Stages

In the first stage, we initially observe the rocket falling due to gravity and leaving the bounds, ending the episode. At around $3{,}750$ iterations, we can see the agent beginning to control the rocket, and the

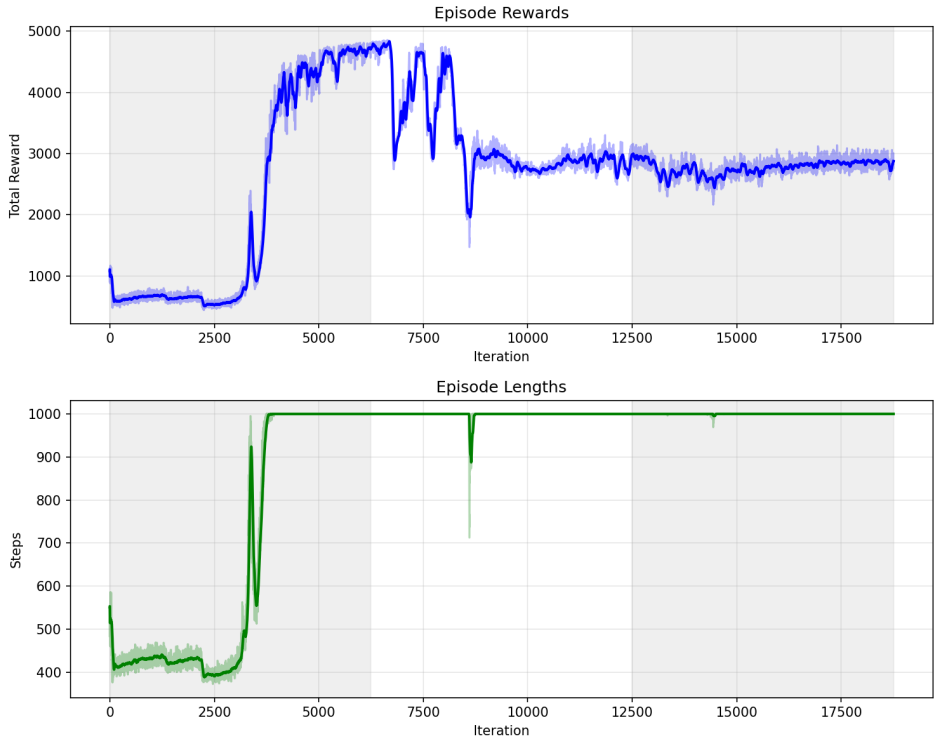

Figure 1: Training curves over 300,000 episodes in batches of 16. Shading alternates between and indicates curriculum stages.

average episodic reward rises to near 5,000. In the early episodes of stage two, with the introduction of light perturbations, we observe a dip in the average reward. Starting in stage three, under full perturbation the agent stabilizes, and the average reward stays moderately consistent throughout the stage.

3.2 Final Performance

By the end of training, the agent achieves an average episodic reward of 2,774.3 throughout the third stage, which is higher than 2,173.6 the average episodic reward of the first stage. However, during the third stage the agent is still not able to achieve as high of an average reward as in the second half of the first stage, since perturbations limit the maximum potential reward. Interestingly, the second stage has the highest average reward of 3,263.0.

4 Discussion

4.1 Generalized Advantage Estimation

Using GAE with $\lambda = 0.95$, the agent was able to stabilize during all stages. Comparatively, with $\lambda = 1$, the average rewards are much lower, and qualitatively speaking, the agent was unable to stabilize the rocket in any of the stages.

4.2 Curriculum Learning

The curriculum used in this experiment is only viable as the tasks of the earlier stages are a subset of the tasks in the later stages, i.e. hovering a rocket at a point is a subset of bringing the rocket to that point and then hovering it.

4.3 Reward Design

The critical feature of this reward function that allowed the agent to effectively use it to represent the value of a point in the state space is it being differentiable everywhere.

4.4 Future Work

Natural extensions of this project include the following: (1) creating a rocket landing environment with a target and extending the curriculum to include this training; (2) increasing the maximum perturbation and height values for the hovering and landing environments respectively; (3) more closely simulating a real physical environment, for example by adding air resistance, wind noise, a minimum throttle value, and reducing to partial observability; (4) further extending the landing environment to include orbital mechanics and training the agent to land from orbit around a celestial mass.

References

J. Schulman, P. Moritz, S. Levine, M. Jordan, and P. Abbeel. High-dimensional continuous control using generalized advantage estimation. In *International Conference on Learning Representations (ICLR)*, 2016.

R. J. Williams. Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine Learning*, 8(3):229–256, 1992.

A Appendix

A.1 Reward Structure

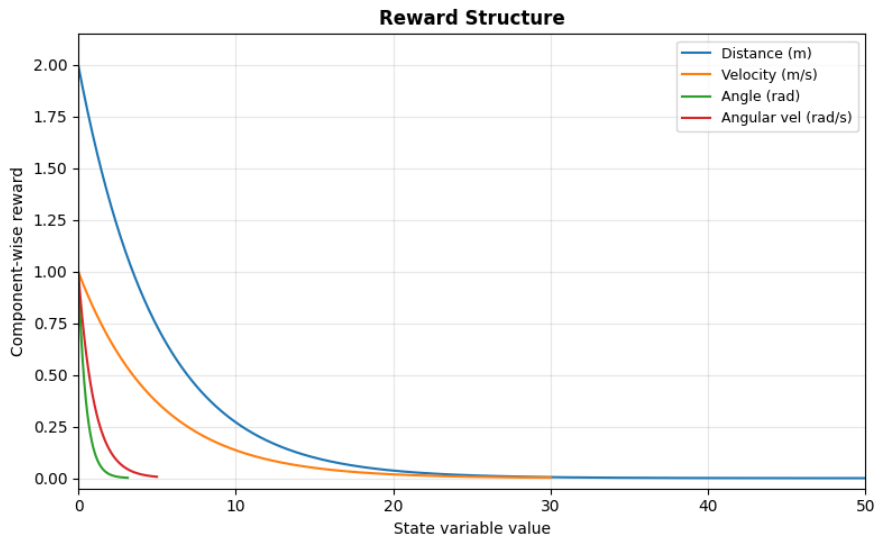

Figure 2: Reward as a function of respective state variable.

A.2 Difficulty Coefficient

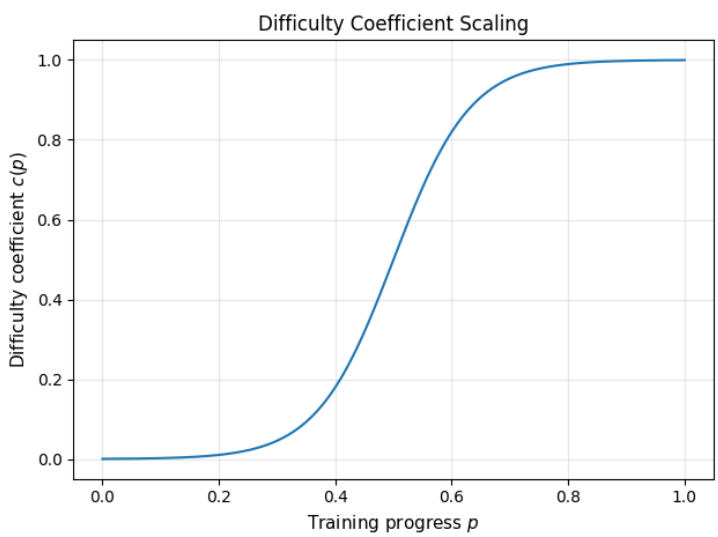

Figure 3: Sigmoid difficulty schedule $c(p) = \frac{1}{1+e^{-e^{\mathcal{E}}(p-1/2)}}$ used in Stage 2 of the curriculum.

A.3 Hyperparameter Summary

Hyperparameter	Value
Discount factor γ	0.99
GAE λ	0.95
Policy learning rate	3×10^{-4}
Value learning rate	1×10^{-3}
Optimizer	Adam
Batch size (parallel episodes)	16
Policy hidden layers	[256, 128]
Value hidden layers	[256, 128]
Activation function	$\tanh$
Max steps per episode	1,000
Time step Δt	0.02 s
Curriculum stages	$3 \times 100{,}000$ episodes

Table 1: Hyperparameters used for training.